8-4 · Streamlit 기반의 챗봇 인터페이스 개발 · 이론 1H

Streamlit 동작 원리와 챗 UI 구성 요소

스크립트가 매번 다시 도는 화면에서 대화 남기기

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① Streamlit 이 상호작용마다 스크립트를 다시 실행하는 원리를 안다

② 일반 변수와 session_state 의 차이를 설명한다

③ @st.cache_data 와 @st.cache_resource 를 구별한다

④ 챗 컴포넌트 여섯 개로 오늘의 뼈대를 그린다

다음 1.5H 실습에서 이 뼈대로 에코 챗봇을 만들고 루프를 뺀 사본도 띄워 봅니다

1. 재실행 모델

상호작용 한 번 = 스크립트 전체를 다시 실행

입력·클릭 →	다시 실행 →	일반 변수 →	session_state
버튼 · chat_input 상호작용 발생	스크립트를 처음부터 끝까지	매번 초기화 0 으로 돌아감	재실행에도 남는다

장점: 상태 관리가 단순해 분석 도구를 몇 줄로 · 단점: 무거운 일(모델 로드·DB 접속)이 매번 반복 → 캐시로 막는다

화면을 「다시 그리는」 것이 아니라 「코드를 다시 실행」 한다 — 모르면 「왜 대화가 한 번만 보이고 사라지지?」에서 헤맨다

같은 버튼인데 — 1 에서 멈추는 것과 쌓이는 것

count = 0 — 일반 변수

재실행마다 0 으로 돌아가 언제나 1 이 찍힌다

- count = 0 이 매번 다시 실행된다
- 버튼 → count += 1 → 1
- st.write(count) 는 절대 2 가 안 된다

VS

st.session_state.n — 저장 공간

재실행에도 지워지지 않아 1, 2, 3 으로 쌓인다

- if "n" not in st.session_state: 로 한 번만 초기화
- 버튼 → st.session_state.n += 1
- st.write 가 1, 2, 3 … 으로 는다

session_state — 재실행에도 남는 저장 공간

1. **지우기 전까지 남는 저장 공간**
   재실행에도 지워지지 않는다 — 대화 이력은 반드시 여기에
2. **브라우저 탭(세션) 단위**
   새로고침하면 사라지고, 다른 사람의 화면과도 분리된다
3. **초기화는 if 로 감싼다**
   if "키" not in st.session_state: — 안 하면 재실행마다 비워진다
4. **서버 메모리에 있다**
   서버를 재시작하면 사라진다 — 오래 남길 것은 DB 에

오래 남길 것은

DB 에 저장

캐시 두 가지 — 값과 연결을 나눠 놓은 이유

@st.cache_data — 값(데이터)

CSV 읽기 결과 · API 응답처럼 복사해도 되는 것

- 함수가 돌려준 값을 저장
- 캐시 키는 함수 인자 — 같으면 저장된 것을
- 복사해도 문제없는 데이터

VS

@st.cache_resource — 연결·객체

DB 클라이언트 · 모델처럼 복사하면 안 되는 것

- Qdrant 접속 · get_emb · get_llm 을 한 번만
- 안 쓰면 재실행마다 접속·모델을 새로
- 값은 복사해도 되지만 연결은 안 된다

챗 컴포넌트 여섯 개 — 함수와 하는 일

함수	하는 일
st.chat_input(...)	아래 고정 입력창 · 보내면 문자열
st.chat_message("user")	사용자 말풍선 컨테이너
st.chat_message("assistant")	어시스턴트 말풍선 컨테이너
st.markdown(text)	말풍선 안에 내용을 쓴다
st.expander("근거")	접었다 펼 수 있는 영역
st.write_stream(...)	글자가 흘러나오게 표시

입력창은 화면 아래에 고정되고, 말풍선은 role 에 따라 user · assistant 로 구분해 그린다

5. 오늘의 뼈대

뼈대 네 단계 — 가장 흔한 실수는 2번 누락

1. **1. 이력 저장소 만들기**
   session_state 에 messages 가 없으면 만든다
2. **2. 이력 전체를 다시 그린다**
   재실행마다 — 이걸 빼면 이전 대화가 화면에서 사라진다
3. **3. chat_input 으로 입력 받기**
   보내면 문자열이 온다
4. **4. 입력이 있으면**
   사용자 메시지 추가·그림 → 답 생성 → 답 추가·그림

답은 오늘

에코 →

나중에 RAG

6. 주소가 달라지는 이유

같은 Qdrant 인데 부르는 이름이 다르다

8-3 · Airflow 태스크 안

컨테이너끼리는 서비스 이름으로 부른다

- Airflow 태스크 안에서 실행
- QDRANT_URL=http://qdrant:6333
- .env 가 넘겨 준다

VS

오늘 · 호스트 터미널

챗봇은 컨테이너 밖(호스트)에서 실행된다

- 내 PC 터미널에서 streamlit run
- QDRANT_URL 없음 → 기본값
- http://localhost:6333

정리

이 1시간에서 꼭 가져갈 것

1. Streamlit 은 상호작용마다 스크립트를 처음부터 다시 실행한다
2. 일반 변수는 매번 초기화되므로 이력은 session_state 에 쌓는다
3. 재실행마다 이력 전체를 다시 그려야 대화가 유지된다
4. 연결·모델은 @st.cache_resource 로 한 번만 만든다
5. 챗봇은 호스트에서 돌므로 Qdrant 주소는 localhost 다

8-4 · 이론 1H 마무리

다음은 챗봇 화면 구현 실습 (1.5H)

에코 챗봇으로 뼈대를 익히고, 다시 그리는 루프를 뺀 사본으로 재실행 모델을 눈으로 확인합니다

- ✓ **rerun_demo.py 로 재실행 눈으로 확인**
- ✓ **chat_echo.py 에코 챗봇 만들기**
- ✓ **루프를 뺀 사본으로 대화가 사라지는 것 재현**
- ✓ **사이드바·스트리밍·캐시 붙이기**

준비할 것

실습점검 1~3단계 · lecture_kb 11